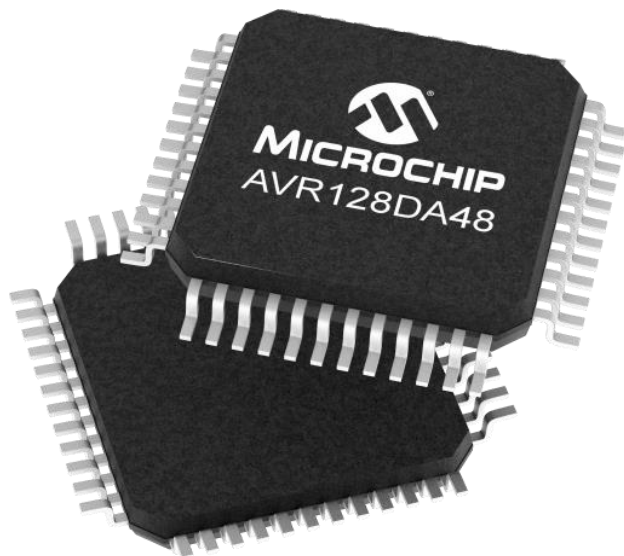
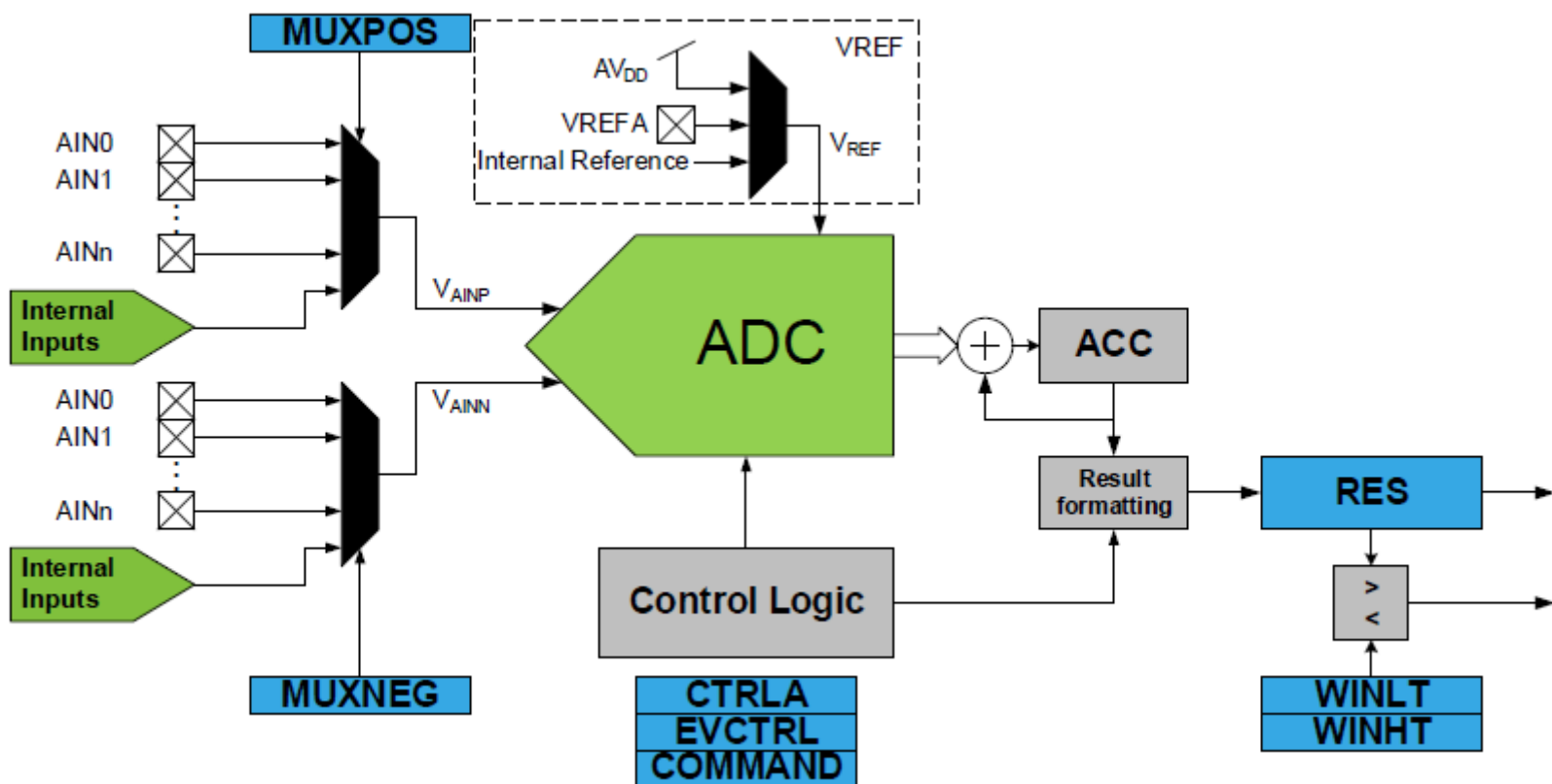


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 17 - ADC Fundamentals



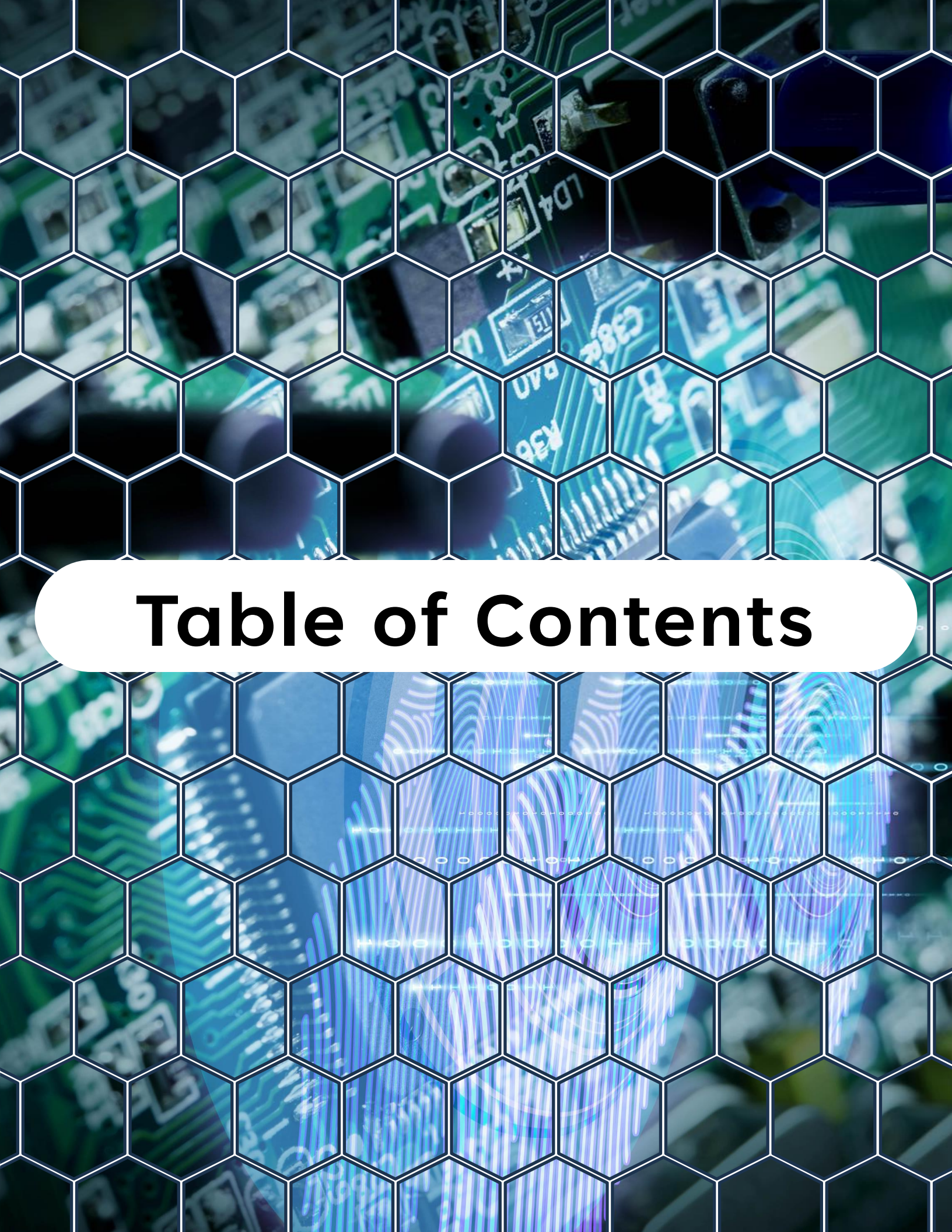
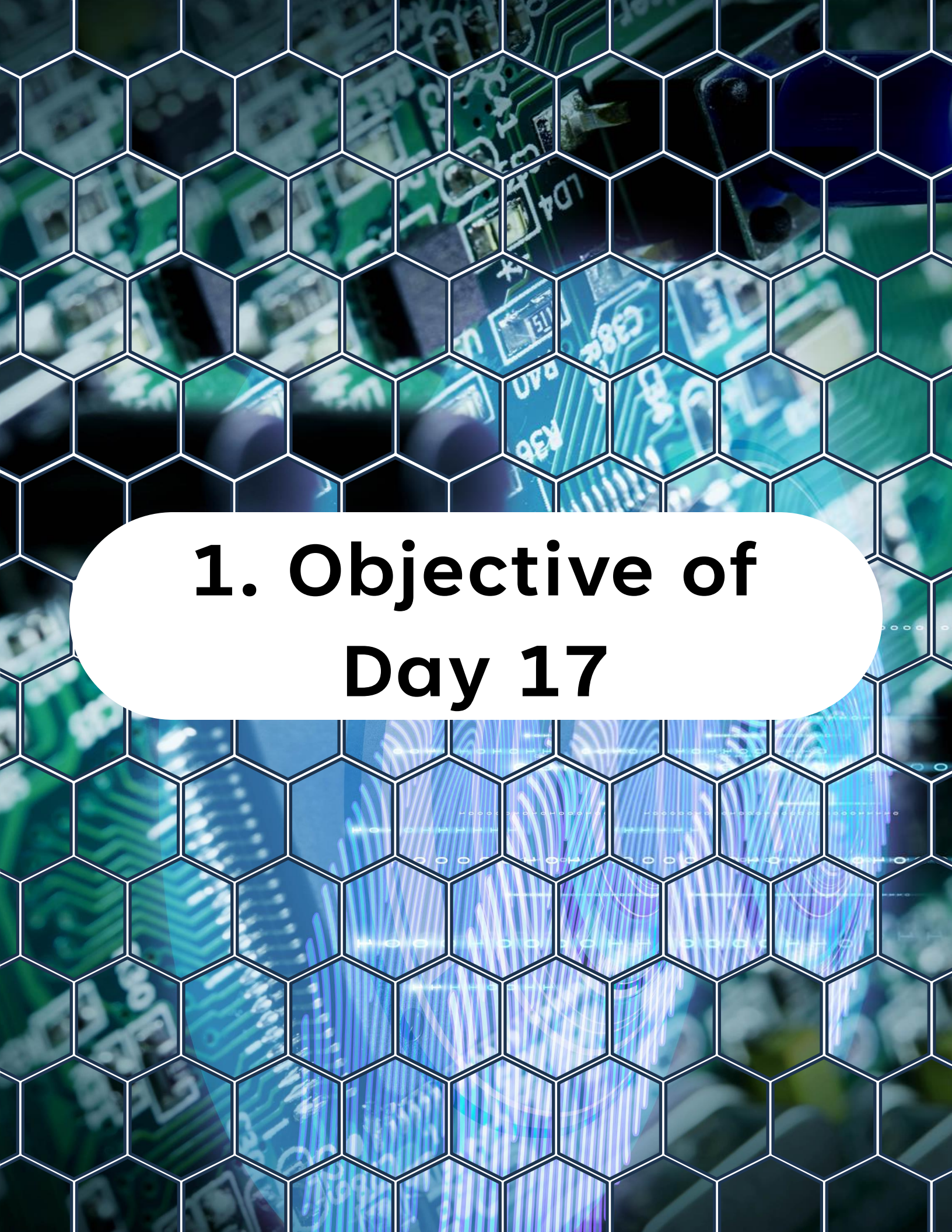


Table of Contents

Table of Contents

1. Objective of Day 17
2. ADC Architecture
3. Reference Selection
4. Resolution and Sampling Time
5. Basic ADC Configuration
6. Performing a Single Conversion
7. Lab - Read Potentiometer Value
8. What You Should Observe
9. Common Mistakes
10. What You Should Understand Now
11. Preview of Day 18



1. Objective of Day 17

1. Objective of Day 17

Today we move into the most important analog peripheral in embedded systems:

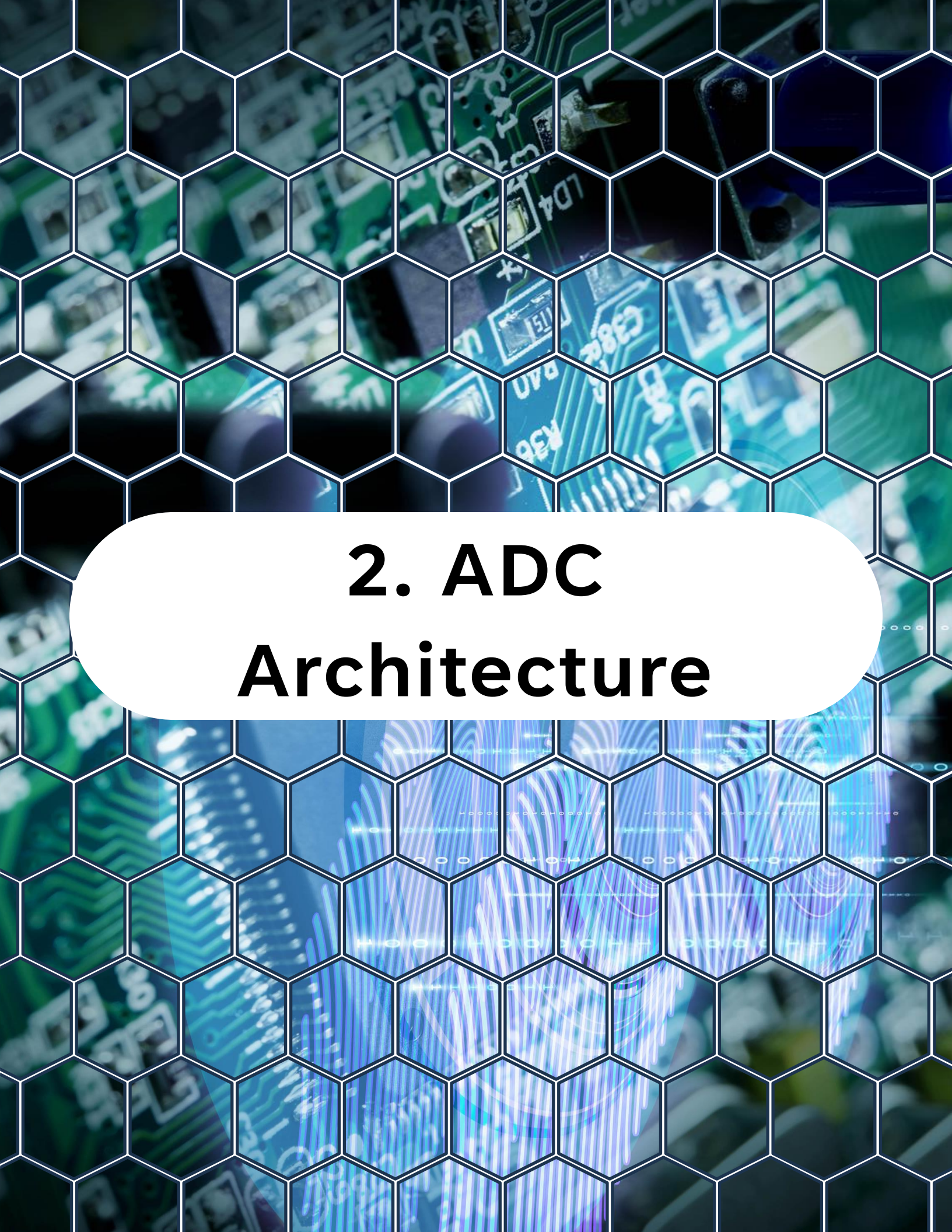
The **Analog-to-Digital Converter (ADC)**.

You already know how to generate digital signals and control timers. Now we will learn how to measure the real world.

By the end of today, you will:

- Understand ADC architecture on the AVR128DA48
- Select and configure voltage references
- Control resolution and sampling time
- Perform a single conversion
- Read a potentiometer value in a clean, register-level lab

No libraries. No HAL. Just the ADC registers.



2. ADC Architecture

2. ADC Architecture

The **Analog-to-Digital Converter (ADC)** is a 12-bit Successive Approximation Register (SAR)

ADC, with a sampling

rate up to 130 ksps at 12-bit resolution.

Key characteristics:

- Resolution: up to 12 bits
- Single-ended input mode
- Multiple analog input channels (AINx pins)
- Configurable voltage reference
- Optional hardware accumulation of Up to 128 Samples per Conversion
- Event-triggered capability
- Free-Running and Single Conversion
- Multiple Voltage Reference Options

2. ADC Architecture

How a SAR ADC Works

The ADC compares the input voltage against an internal reference.

At 12-bit resolution:

Digital range = 0 to 4095

If $V_{REF} = 4.096V$:

$1 \text{ LSB} = 4.096V / 4096 \approx 1 \text{ mV}$

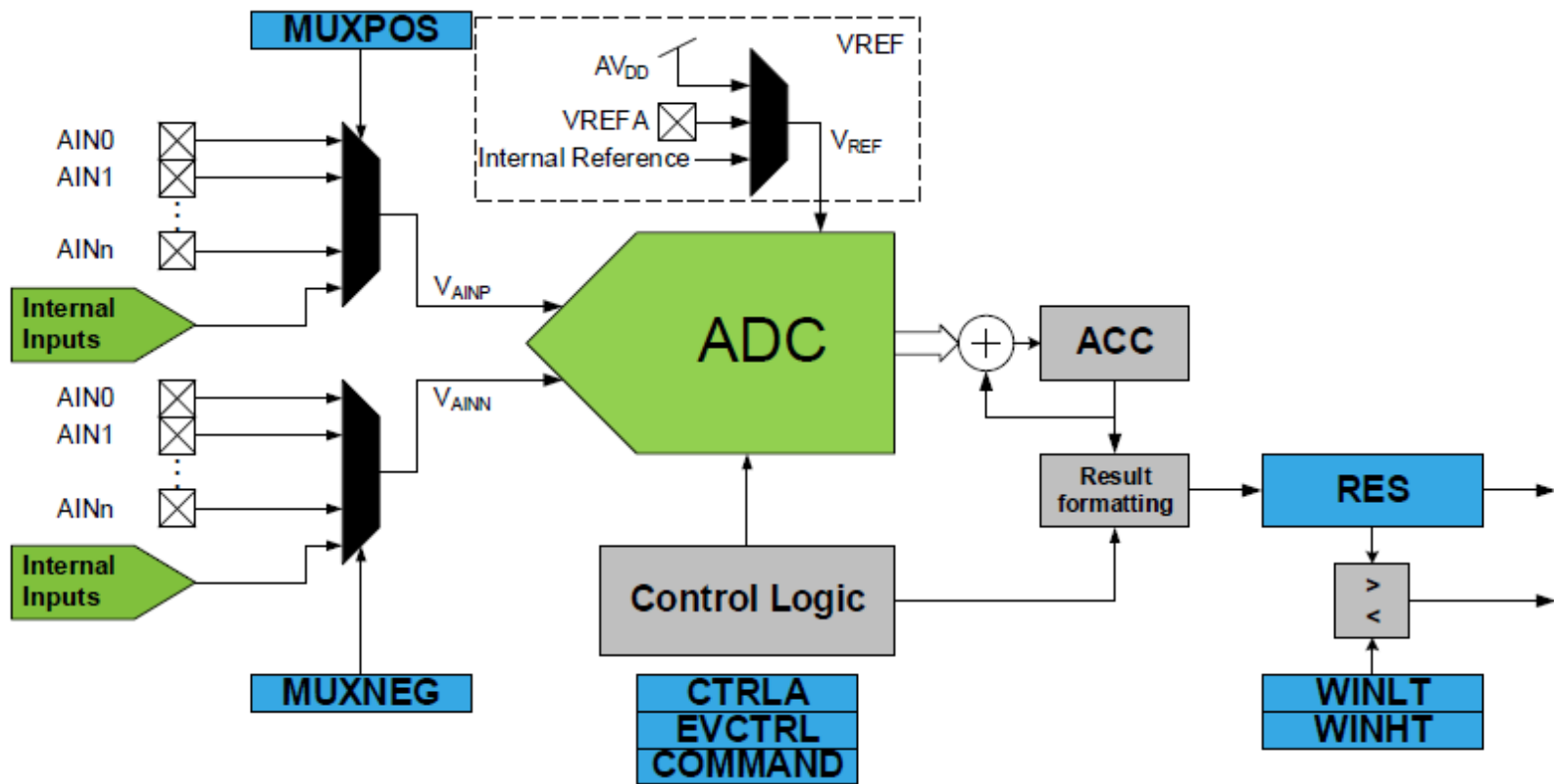
The ADC result is:

$$\text{ADC_result} = (V_{in} / V_{REF}) * 4095$$

This means the reference you choose directly affects measurement accuracy.

2. ADC Architecture

ADC Block Diagram





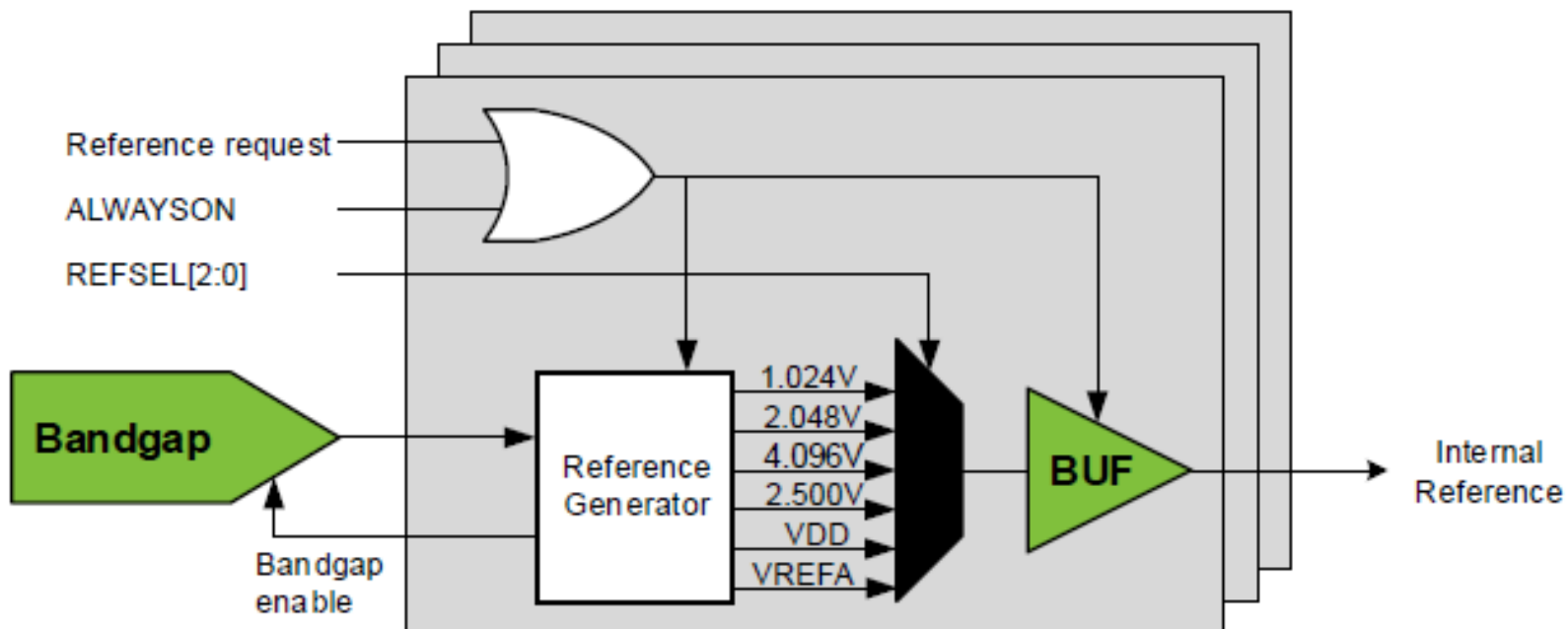
3. Reference Selection

3. Reference Selection

The ADC does not measure absolute voltage. It measures relative to its reference.

Common reference options:

- VDD
- Internal 1.024V
- Internal 2.048V
- Internal 4.096V
- Internal 2.500V
- External reference pin



3. Reference Selection

Why Reference Choice Matters

Example:

- If $V_{REF} = 4.096V$

Resolution $\approx 1 \text{ mV}$ per step

- If $V_{REF} = 1.024V$

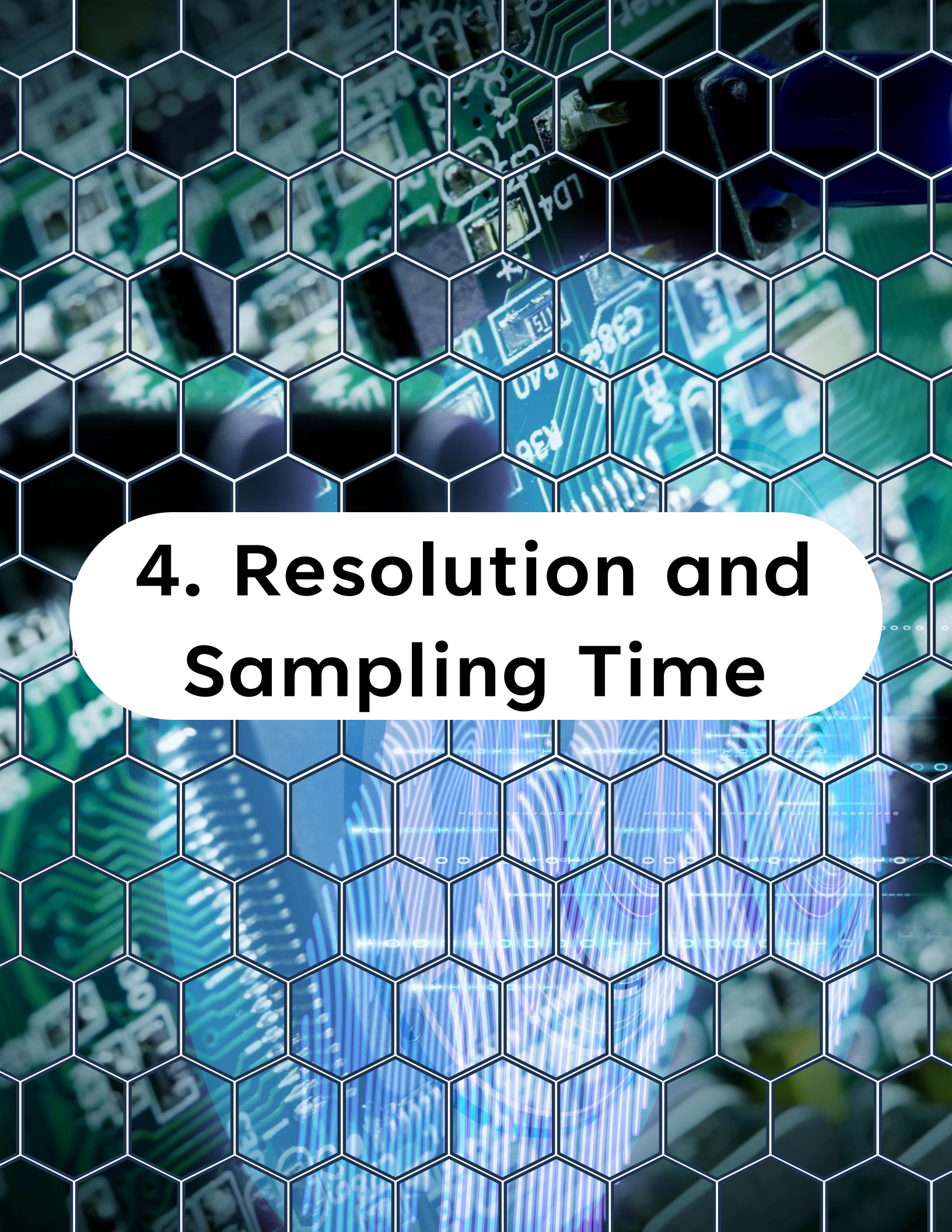
Resolution $\approx 0.25 \text{ mV}$ per step

Lower reference -> better resolution

But input voltage must stay below V_{REF} .

Always choose reference based on:

- Expected signal range
- Required precision



4. Resolution and Sampling Time

4. Resolution and Sampling Time

Resolution

The AVR DA ADC supports:

- 12-bit mode
- 10-bit mode (optional configuration)

Higher resolution:

- More precise
- Slightly longer conversion time

Sampling Time

Before conversion begins, the ADC sampling capacitor must charge.

If:

- Source impedance is high
- Sampling time is too short

4. Resolution and Sampling Time

You get inaccurate readings.

The ADC prescaler controls:

- ADC clock frequency
- Conversion timing

Best practice:

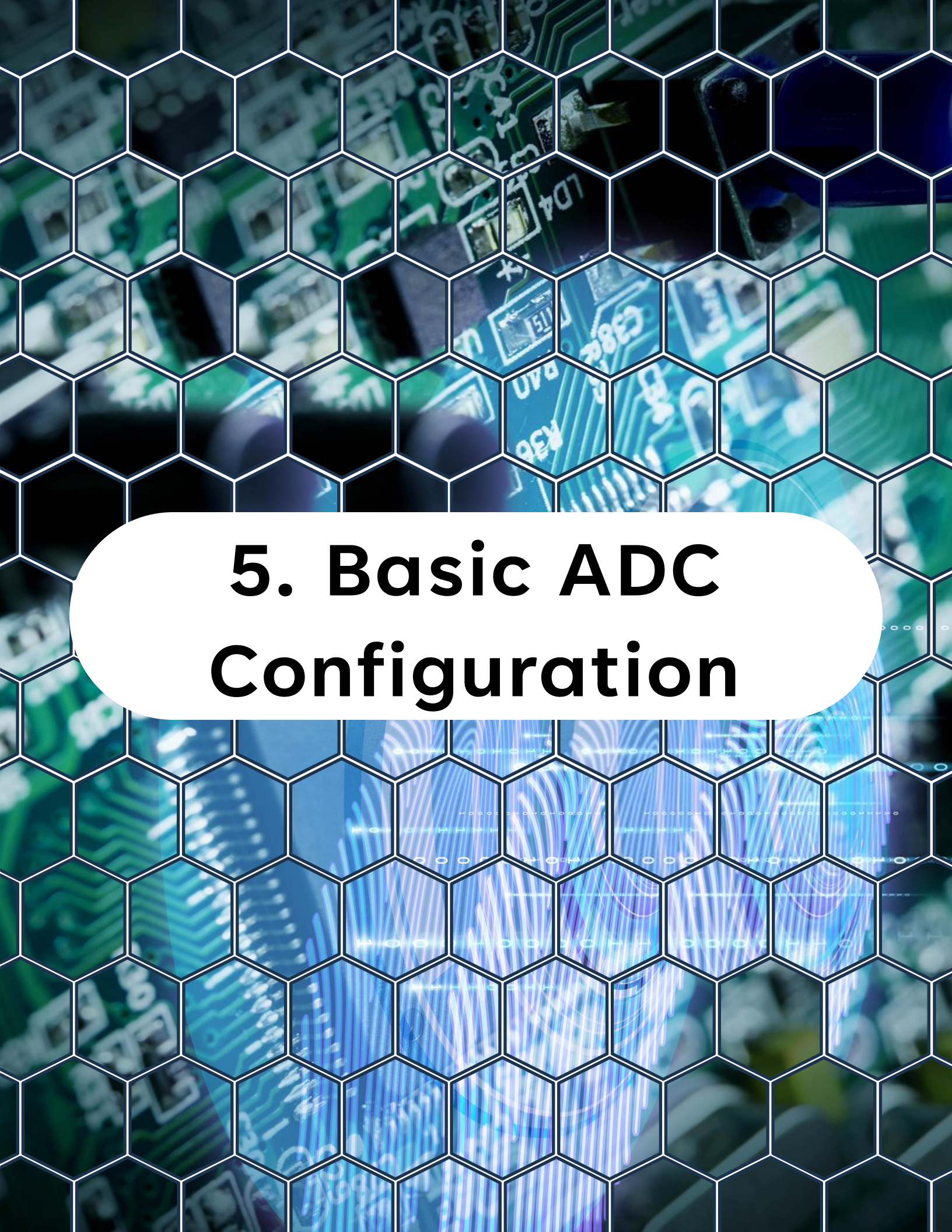
Keep ADC clock between 50 kHz and 1 MHz.

Example:

$F_{\text{CPU}} = 24 \text{ MHz}$

Prescaler = DIV16

ADC clock = 1.5 MHz (acceptable)

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the section title in bold black text.

5. Basic ADC Configuration

5. Basic ADC Configuration

We configure:

- Reference = 4.096V
- Prescaler = DIV16
- Input channel = AIN0
- Single conversion mode

Minimal ADC Initialization Example

```
1  /**
2   * @brief Initialize ADC0 for single-ended 12-bit conversion.
3   */
4  static void ADC0_init(void)
5  {
6      // Disable ADC before configuration
7      ADC0.CTRLA = 0;
8
9      // Select reference
10     // Enable reference always ON for ADC0
11     // Internal 4.096V reference
12     VREF.ADC0REF = 1 << VREF_ALWAYS_ON_bp
13                 | VREF_REFSEL_4V096_gc;
14
15     // ADC input pin 0
16     ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
```


5. Basic ADC Configuration

```
1  /**
2   * @brief Initialize ADC0 for single-ended 12-bit conversion.
3   */
4  static void ADC0_init(void)
5  {
6      // Disable ADC before configuration
7      ADC0.CTRLA = 0;
8
9      // Select reference
10     // Enable reference always ON for ADC0
11     // Internal 4.096V reference
12     VREF.ADC0REF = 1 << VREF_ALWAYS_ON_bp
13                 | VREF_REFSEL_4V096_gc;
14
15     // ADC input pin 0
16     ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
17
18     // No accumulation
19     ADC0.CTRLB = ADC_SAMPNUM_NONE_gc;
20
21     // Prescaler = CLK_PER / 16
22     ADC0.CTRLC = ADC_PRESC_DIV16_gc;
23
24     // 12-bit mode
25     ADC0.CTRLA |= ADC_RESSEL_12BIT_gc;
26
27     // Single-ended conversion mode
28     ADC0.CTRLA |= 0 << ADC_CONVMODE_bp;
29
30     // Enable ADC
31     ADC0.CTRLA |= 1 << ADC_ENABLE_bp;
32 }
```



6. Performing a Single Conversion

6. Performing a Single Conversion

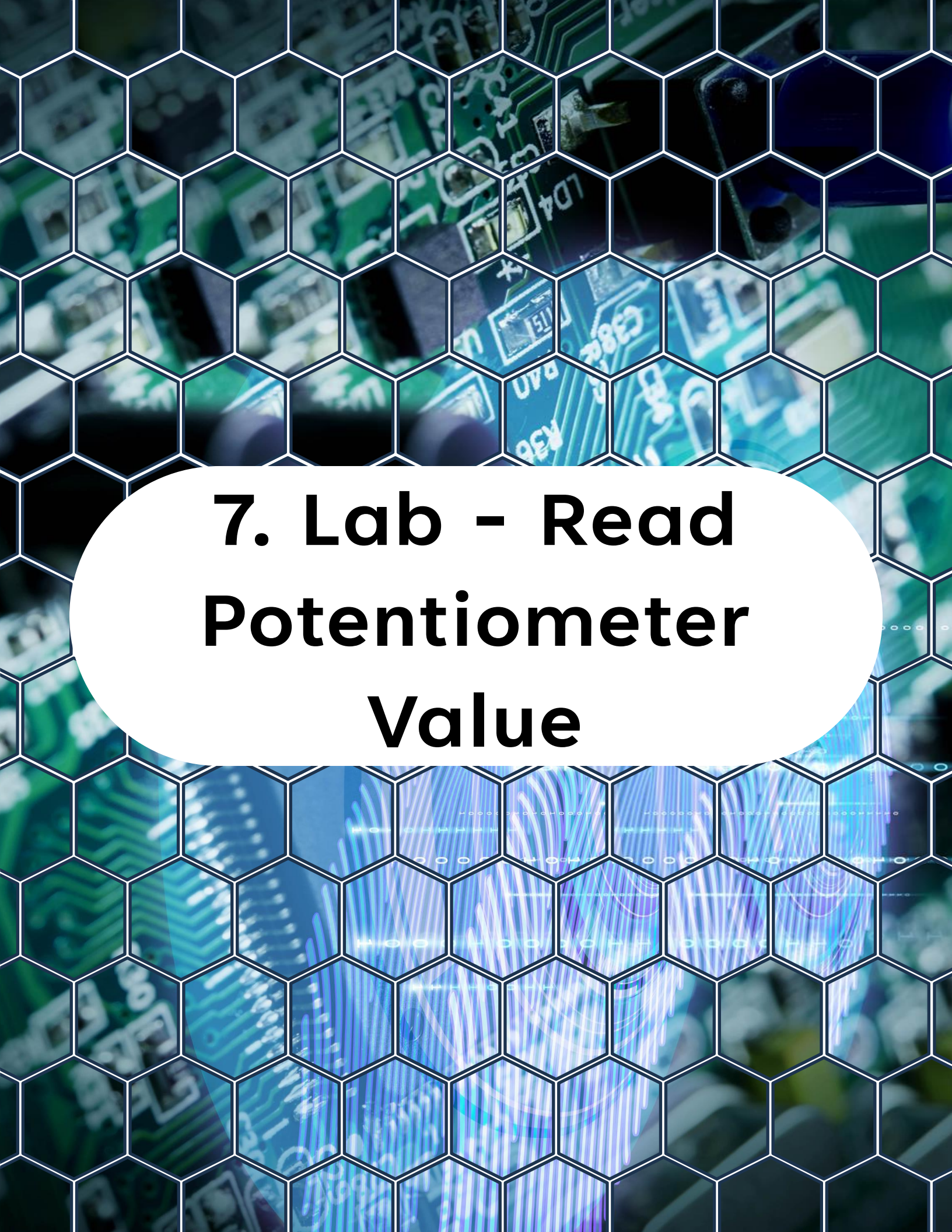
Steps:

1. Select input channel
2. Start conversion
3. Wait for result
4. Read result

```
1  /**
2   * @brief Perform single ADC conversion on AIN0.
3   */
4  static uint16_t ADC0_read(void)
5  {
6      // Select channel
7      ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
8
9      // Start conversion
10     ADC0.COMMAND = ADC_STCONV_bm;
11
12     // Wait for result ready
13     while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
14     {
15         // Do nothing
16     }
```


6. Performing a Single Conversion

```
1  /**
2   * @brief Perform single ADC conversion on AIN0.
3   */
4  static uint16_t ADC0_read(void)
5  {
6      // Select channel
7      ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
8
9      // Start conversion
10     ADC0.COMMAND = ADC_STCONV_bm;
11
12     // Wait for result ready
13     while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
14     {
15         // Do nothing
16     }
17
18     // Clear flag
19     ADC0.INTFLAGS = ADC_RESRDY_bm;
20
21     // Return result
22     return ADC0.RES;
23 }
```



7. Lab - Read Potentiometer Value

7. Lab - Read Potentiometer Value

Hardware Setup

- Connect potentiometer:
 - One end to VDD
 - Other end to GND
 - Wiper to AIN0
- Use internal 4.096V reference

When you rotate the pot:

Voltage at AIN0 changes between 0V and VDD.

ADC reading should change accordingly.

Complete Lab Example

```
1  /**
2   * @file main.c
3   * @brief ADC lab: Read potentiometer value on AIN0.
4   */
5  #include <avr/io.h>
```


7. Lab - Read Potentiometer Value

Complete Lab Example

```
1  /**
2   * @file main.c
3   * @brief ADC lab: Read potentiometer value on AIN0.
4   */
5  #include <avr/io.h>
6  #include <stdint.h>
7
8  /**
9   * @brief Initialize ADC0 for single-ended 12-bit conversion.
10  */
11 static void ADC0_init(void)
12 {
13     // Disable ADC before configuration
14     ADC0.CTRLA = 0;
15
16     // Select reference
17     // Enable reference always ON for ADC0
18     // Internal 4.096V reference
19     VREF.ADC0REF = 1 << VREF_ALWAYS_ON_bp
20                  | VREF_REFSEL_4V096_gc;
21
22     // ADC input pin 0
23     ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
24
25     // No accumulation
26     ADC0.CTRLB = ADC_SAMPNUM_NONE_gc;
27
28     // Prescaler = CLK_PER / 16
29     ADC0.CTRLC = ADC_PRESC_DIV16_gc;
30
```

7. Lab - Read Potentiometer Value

```
31 // 12-bit mode
32 ADC0.CTRLA |= ADC_RESSEL_12BIT_gc;
33
34 // Single-ended conversion mode
35 ADC0.CTRLA |= 0 << ADC_CONVMODE_bp;
36
37 // Enable ADC
38 ADC0.CTRLA |= 1 << ADC_ENABLE_bp;
39 }
40
41 /**
42  * @brief Perform single ADC conversion on AIN0.
43  */
44 static uint16_t ADC0_read(void)
45 {
46     // Select channel
47     ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
48
49     // Start conversion
50     ADC0.COMMAND = ADC_STCONV_bm;
51
52     // Wait for result ready
53     while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
54     {
55         // Do nothing
56     }
57
58     // Clear flag
59     ADC0.INTFLAGS = ADC_RESRDY_bm;
60
61     // Return result
62     return ADC0.RES;
63 }
64
```

7. Lab - Read Potentiometer Value

```
65 /**
66  * @brief Program entry point.
67  */
68 int main(void)
69 {
70     uint16_t adc_value;
71
72     // Initialize ADC and continuously read potentiometer value.
73     ADC0_init();
74
75     while (1)
76     {
77         adc_value = ADC0_read();
78
79         /* Place breakpoint here and observe adc_value */
80     }
81 }
```




8. What You Should Observe

8. What You Should Observe

Rotate the potentiometer:

- ADC value changes smoothly
- Minimum ≈ 0
- Maximum ≈ 4095 (if $V_{DD} \approx V_{REF}$)

If using 4.096V reference and $V_{DD} = 5V$:

Maximum reading will be slightly below 4095

Because input exceeds reference at upper end.

That is expected.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', 'U7A', and 'K38'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the section title in bold black text.

9. Common Mistakes

9. Common Mistakes

1. Forgetting to enable ADC
2. Using wrong reference selection
3. Forgetting to clear RESRDY flag
4. Using too high ADC clock
5. Not allowing enough sampling time

Always verify:

- Reference selection
- Input pin configuration
- Correct MUXPOS



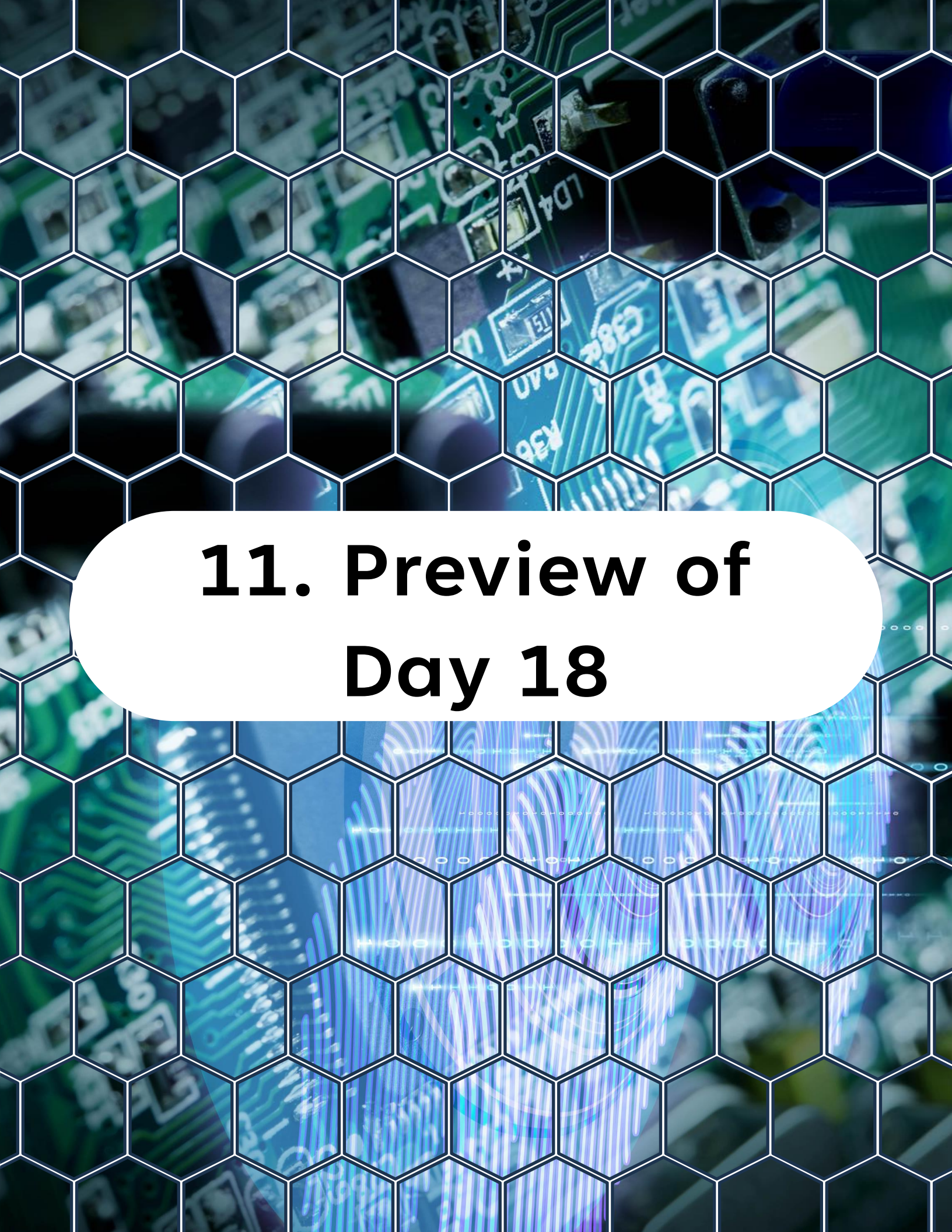
10. What You Should Understand Now

10. What You Should Understand Now

You now understand:

- SAR ADC architecture
- Reference selection tradeoffs
- Resolution meaning
- Sampling timing considerations
- Basic single conversion flow

You can now measure real-world analog signals.



11. Preview of Day 18

11. Preview of Day 18

Tomorrow we go deeper:

- Window comparator mode
- Accumulation and averaging
- Interrupt-driven sampling

We move from "read a value" to "react to conditions."

That is where embedded systems become intelligent.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>